

✓ Real-time Audio Processing: Swahili Speech Analysis

This notebook implements a comprehensive audio processing pipeline using the Mozilla Common Voice Swahili dataset, following the CRISP-DM methodology. We'll cover speech-to-text conversion, sentiment analysis, and text summarization with state-of-the-art models.

Table of Contents

1. [Library Imports and Setup](#)
2. [Business Understanding](#)
3. [Data Understanding](#)
4. [Data Preparation](#)
5. [Modeling](#)
6. [Evaluation](#)
7. [Deployment](#)
8. [Feedback Loop and Model Refinement](#)

✓ 1. Library Imports and Setup

First, we'll import all necessary libraries and explain their purposes in our project.

```
# Core ML and Data Processing
!pip install torch
!pip install tensorflow
!pip install transformers
!pip install librosa
!pip install numpy
!pip install pandas
!pip install datasets
```

```
# Visualization
!pip install matplotlib
!pip install seaborn
```

```
# NLP and Speech Processing
!pip install spacy
!pip install speechbrain
!pip install rouge-score
```

```
# Word Processing
!pip install jiwer
```

```
# Parallel Processing
!pip install tqdm
```

```
# Web Interface
!pip install streamlit
```

```
# Vector Search
!pip install faiss-cpu
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.11/dist-packages (2.5.1+cu121)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch) (3.16.1)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.11/dist-packages (from torch) (4.12.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch) (3.4.2)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.11/dist-packages (from torch) (3.1.5)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch) (2024.10.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.1.105 in /usr/local/lib/python3.11/dist-packages (from torch) (12.1.105)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.1.105 in /usr/local/lib/python3.11/dist-packages (from torch) (12.1.105)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.1.105 in /usr/local/lib/python3.11/dist-packages (from torch) (12.1.105)
Requirement already satisfied: nvidia-cudnn-cu12==9.1.0.70 in /usr/local/lib/python3.11/dist-packages (from torch) (9.1.0.70)
Requirement already satisfied: nvidia-cublas-cu12==12.1.3.1 in /usr/local/lib/python3.11/dist-packages (from torch) (12.1.3.1)
Requirement already satisfied: nvidia-cufft-cu12==11.0.2.54 in /usr/local/lib/python3.11/dist-packages (from torch) (11.0.2.54)
Requirement already satisfied: nvidia-curand-cu12==10.3.2.106 in /usr/local/lib/python3.11/dist-packages (from torch) (10.3.2.106)
Requirement already satisfied: nvidia-cusolver-cu12==11.4.5.107 in /usr/local/lib/python3.11/dist-packages (from torch) (11.4.5.107)
Requirement already satisfied: nvidia-cuspars-cu12==12.1.0.106 in /usr/local/lib/python3.11/dist-packages (from torch) (12.1.0.106)
```

```

Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in /usr/local/lib/python3.11/dist-packages (from torch) (2.21.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.1.105 in /usr/local/lib/python3.11/dist-packages (from torch) (12.1.105)
Requirement already satisfied: triton==3.1.0 in /usr/local/lib/python3.11/dist-packages (from torch) (3.1.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-packages (from torch) (1.13.1)
Requirement already satisfied: nvidia-nvjitlink-cu12 in /usr/local/lib/python3.11/dist-packages (from torch) (1.13.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2->torch) (3.0.2)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.11/dist-packages (2.17.1)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (24.12.23)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: h5py>=3.10.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.12.1)
Requirement already satisfied: libclang==13.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: ml-dtypes<0.5.0,>=0.3.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.4.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from tensorflow) (24.2)
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<5.0.0dev,>=3.20.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (4.21.3)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.32.3)
Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from tensorflow) (75.1.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.5.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (4.12.2)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.69.0)
Requirement already satisfied: tensorboard<2.18,>=2.17 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.17.1)
Requirement already satisfied: keras>=3.2.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.5.0)
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.37.0)
Requirement already satisfied: numpy<2.0.0,>=1.23.5 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.26.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from astunparse>=1.6.0->tensorflow) (0.43.0)
Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->tensorflow) (0.0.8)
Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->tensorflow) (0.13.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.1.1)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.18,>=2.17->tensorflow) (3.7)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.18,>=2.17->tensorflow) (0.17.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.18,>=2.17->tensorflow) (3.0.6)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from werkzeug>=1.0.1->tensorboard<2.18,>=2.17->tensorflow) (3.0.2)

```

```
# Core ML and Data Processing
```

```
import torch
import tensorflow as tf
from transformers import AutoModelForSpeechSeq2Seq, AutoProcessor, pipeline
import librosa
import numpy as np
import pandas as pd
from datasets import load_dataset
from torch.utils.data import DataLoader
```

```
# Visualization
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Normalization
```

```
import nltk
#nltk.download('punkt')
#nltk.download('stopwords')
```

```
# NLP and Speech Processing
```

```
import spacy
from speechbrain.inference import EncoderDecoderASR
```

```
# Parallel Processing
```

```
from concurrent.futures import ThreadPoolExecutor
import multiprocessing
```

```
# Web Interface
```

```
import streamlit as st
```

```
# Vector Search
```

```
import faiss
```

```
# Utility
```

```
import os
import json
import time
```

```
from tqdm.notebook import tqdm

print("Libraries imported successfully!")

↵ Libraries imported successfully!
```

Library Purposes

- **torch & tensorflow:** Deep learning frameworks for model development
- **transformers:** Hugging Face's library for state-of-the-art NLP models
- **librosa:** Audio and music processing
- **spacy:** Natural language processing tasks
- **speechbrain:** Speech recognition and processing
- **matplotlib & seaborn:** Data visualization
- **streamlit:** Interactive web application development
- **faiss:** Efficient similarity search for large datasets
- **datasets:** Hugging Face's dataset management

2. Business Understanding

Problem Definition

Our project aims to develop a comprehensive audio processing system for Swahili speech with three main objectives:

1. **Speech-to-Text Conversion:** Accurately transcribe Swahili speech to text
2. **Sentiment Analysis:** Analyze the emotional content of transcribed speech
3. **Text Summarization:** Generate concise summaries of transcribed content

Real-world Applications

- **Education:** Supporting language learning and assessment
- **Business Intelligence:** Analyzing customer feedback in Swahili-speaking regions
- **Media Monitoring:** Processing Swahili broadcast content
- **Healthcare:** Facilitating medical consultations in Swahili-speaking communities
- **Government Services:** Improving accessibility of public services

✓ 3. Data Understanding

Let's load and explore the Mozilla Common Voice Swahili dataset.

```
# Load the dataset
dataset = load_dataset("mozilla-foundation/common_voice_11_0", "sw", split="train", trust_remote_code=True)

# from datasets import load_dataset
# from huggingface_hub import login

# Log in to Hugging Face Hub
# login()

# dataset = load_dataset("mozilla-foundation/common_voice_17_0", "sw", split="train", trust_remote_code=True)
# dataloader = DataLoader(cv_17, batch_size=32)

# Display basic information
print(f"Dataset size: {len(dataset)} samples")
print("\nDataset features:", dataset.features)

# Create a DataFrame for analysis
df = pd.DataFrame({
    'path': dataset['path'],
    'sentence': dataset['sentence'],
    'age': dataset['age'],
    'gender': dataset['gender'],
    'accent': dataset['accent']
})
```

```
})
# Display sample statistics
print("\nDataset Overview:")
print(df.describe())


```

↳

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secre
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

README.md: 100% 14.4k/14.4k [00:00<00:00, 531kB/s]

common_voice_11_0.py: 100% 8.13k/8.13k [00:00<00:00, 300kB/s]

languages.py: 100% 3.44k/3.44k [00:00<00:00, 134kB/s]

release_stats.py: 100% 60.9k/60.9k [00:00<00:00, 2.73MB/s]

n_shards.json: 100% 12.2k/12.2k [00:00<00:00, 418kB/s]

sw_train_0.tar: 100% 907M/907M [00:13<00:00, 76.1MB/s]

sw_dev_0.tar: 100% 352M/352M [00:07<00:00, 44.7MB/s]

sw_test_0.tar: 100% 355M/355M [00:07<00:00, 42.5MB/s]

sw_other_0.tar: 100% 1.28G/1.28G [00:28<00:00, 46.0MB/s]

sw_other_1.tar: 100% 1.27G/1.27G [00:24<00:00, 62.0MB/s]

sw_other_2.tar: 100% 1.19G/1.19G [00:33<00:00, 31.1MB/s]

sw_other_3.tar: 100% 1.24G/1.24G [00:23<00:00, 64.2MB/s]

sw_other_4.tar: 100% 1.26G/1.26G [00:28<00:00, 53.8MB/s]

sw_other_5.tar: 100% 1.28G/1.28G [00:32<00:00, 42.3MB/s]

sw_other_6.tar: 100% 3.30M/3.30M [00:00<00:00, 10.6MB/s]

sw_invalidated_0.tar: 100% 1.44G/1.44G [00:27<00:00, 53.9MB/s]

sw_invalidated_1.tar: 100% 284M/284M [00:06<00:00, 35.0MB/s]

train.tsv: 100% 6.30M/6.30M [00:00<00:00, 73.9MB/s]

dev.tsv: 100% 2.42M/2.42M [00:00<00:00, 71.5MB/s]

test.tsv: 100% 2.40M/2.40M [00:00<00:00, 67.7MB/s]

other.tsv: 100% 55.9M/55.9M [00:00<00:00, 138MB/s]

invalidated.tsv: 100% 11.2M/11.2M [00:00<00:00, 102MB/s]

Generating train split: 26614/0 [00:12<00:00, 2744.96 examples/s]

Reading metadata...: 0it [00:00, ?it/s]
Reading metadata...: 3113it [00:00, 29147.20it/s]
Reading metadata...: 8739it [00:00, 44655.03it/s]
Reading metadata...: 26614it [00:00, 66660.54it/s]
Generating validation split: 10233/0 [00:03<00:00, 3017.85 examples/s]

Reading metadata...: 10233it [00:00, 126111.22it/s]

df.head(3)

↳

	path	sentence	age	gender	accent
0	/root/.cache/huggingface/datasets/downloads/ex...	Uko katika pembe la kusini-mashariki kabisa la...			
1	/root/.cache/huggingface/datasets/downloads/ex...	Ina bandari kubwa na ni kitovu cha biashara ya...			
2	/root/.cache/huggingface/datasets/downloads/ex...	Ndiyo maana upatikanaji wake wakati mwingine i...			

Reading metadata...: 24927it [00:00, 80011.56it/s]

df.describe().T

https://colab.research.google.com/drive/1bldufC7-Uc7r_AVnUMXhh4a8BnK3uLni#printMode=true

4/23



	count	unique		top	freq
path	26614	26614	/root/.cache/huggingface/datasets/downloads/ex...		1
sentence	26614	26614	Uko katika pembe la kusini-mashariki kabisa la...		1
age	26614	6		twenties	13874
gender	26614	3		female	9771
accent	26614	1			26614

Reading metadata...: 1834771t [00:02, 57867.391t/s]

df.dtypes



	0
path	object
sentence	object
age	object
gender	object
accent	object

dtype: object

df.info()



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26614 entries, 0 to 26613
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   path        26614 non-null   object
1   sentence    26614 non-null   object
2   age         26614 non-null   object
3   gender      26614 non-null   object
4   accent      26614 non-null   object
dtypes: object(5)
memory usage: 1.0+ MB
unique      1
```

```
# Display dataset size and features
print(f"Dataset size: {len(dataset)} samples")
```



```
Dataset size: 26614 samples
```

```
print("\nDataset features:", dataset.features)
```



```
Dataset features: {'client_id': Value(dtype='string', id=None), 'path': Value(dtype='string', id=None), 'audio': Audio(sampling_rate=48000, num_samples=1000000)}
```

```
print("\nMissing Values:")
df.isnull().sum()
```



```
Missing Values:
#   Column      Non-Null Count  Dtype
---  -
0   path        26614 non-null   object
1   sentence    26614 non-null   object
2   age         26614 non-null   object
3   gender      26614 non-null   object
4   accent      26614 non-null   object
dtypes: object(5)
memory usage: 1.0+ MB
unique      1
```

We do not have any missing values

df.columns

```
Index(['path', 'sentence', 'age', 'gender', 'accent'], dtype='object')
```

Univariate Analysis

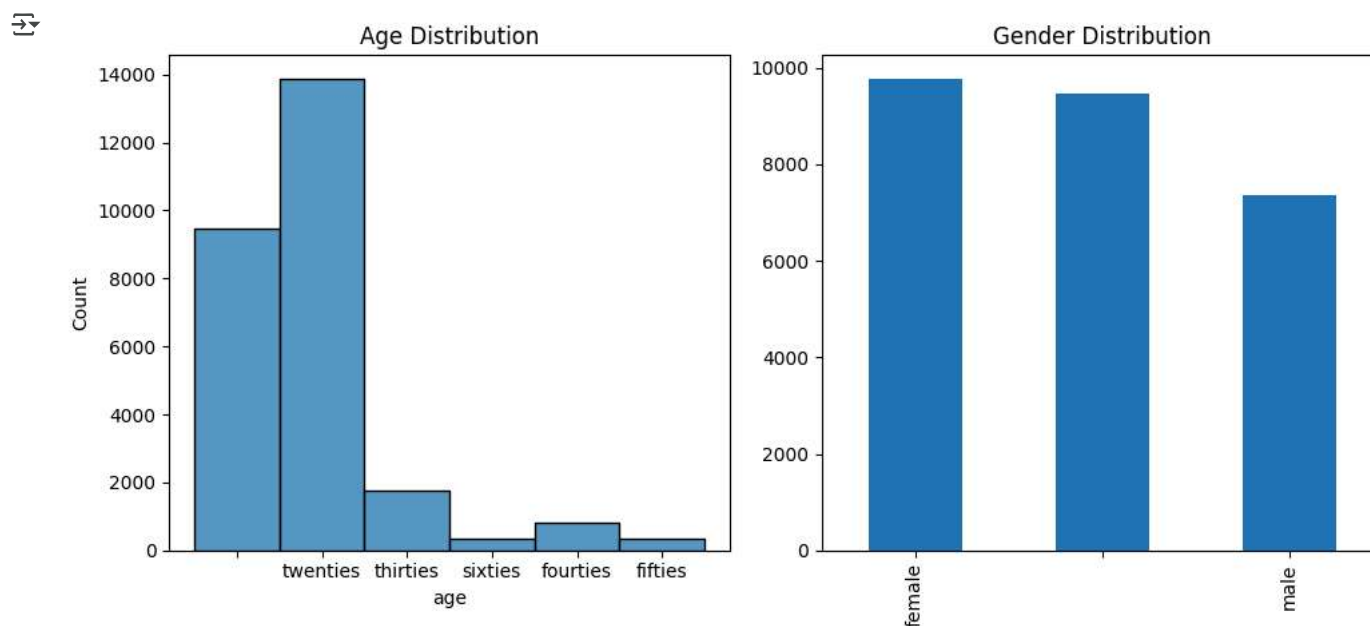
```
# Visualize dataset statistics
plt.figure(figsize=(15, 5))
```

```
# Age distribution
plt.subplot(131)
sns.histplot(df['age'].dropna())
plt.title('Age Distribution')
```

```
# Gender distribution
plt.subplot(132)
df['gender'].value_counts().plot(kind='bar')
plt.title('Gender Distribution')
```

```
# Accent distribution
#plt.subplot(133)
#df['accent'].value_counts().plot(kind='bar')
#plt.title('Accent Distribution')
```

```
plt.tight_layout()
plt.show()
```



```
# Print summary statistics for age, gender, and accent
print("\nAge Statistics:")
print(df['age'].describe())
```

```
print("\nGender Distribution:")
print(df['gender'].value_counts())
```

```
print("\nAccent Distribution:")
print(df['accent'].value_counts())
```

```
Age Statistics:
count      26614
unique         6
top    twenties
freq      13874
Name: age, dtype: object
```

```
Gender Distribution:
gender
female    9771
male      7369
Name: count, dtype: int64
```

```
Accent Distribution:
```

```
accent
26614
Name: count, dtype: int64

# Replace blank spaces with 'unknown'
df['gender'] = df['gender'].str.strip().replace('', 'unknown')

# Replace blank spaces with 'unknown'
df['age'] = df['age'].str.strip().replace('', 'unknown')

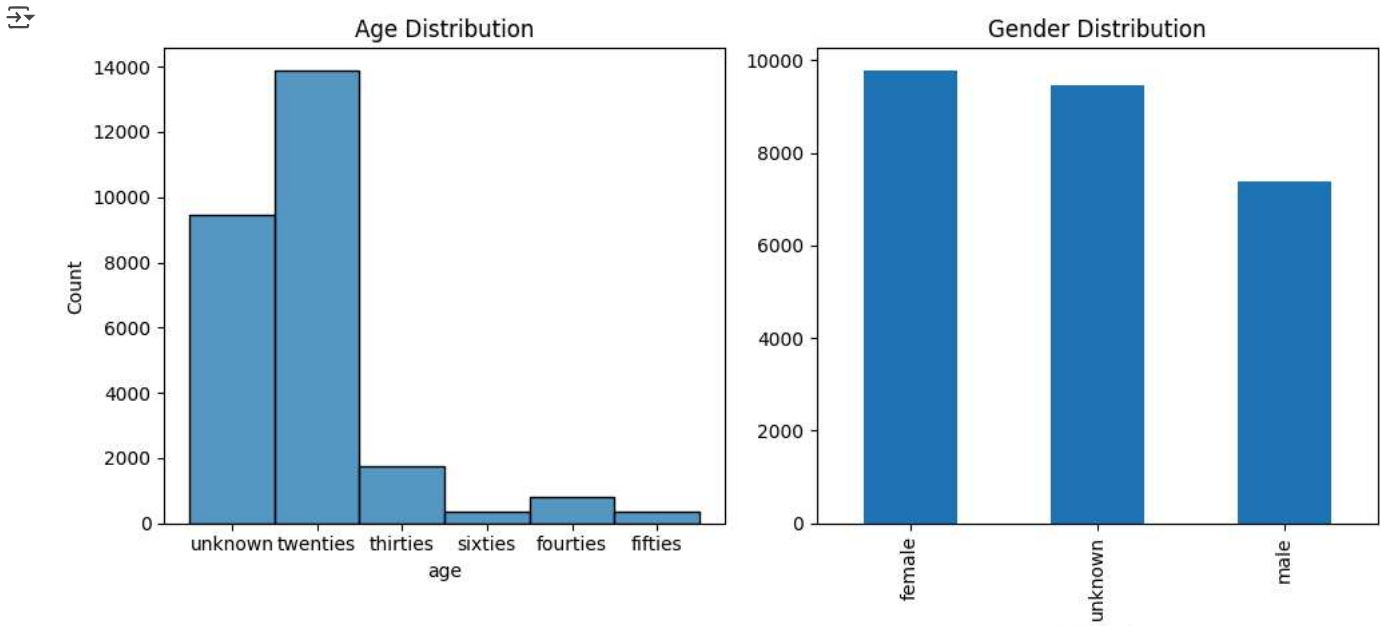
# Visualize dataset statistics
plt.figure(figsize=(15, 5))

# Age distribution
plt.subplot(131)
sns.histplot(df['age'].dropna())
plt.title('Age Distribution')

# Gender distribution
plt.subplot(132)
df['gender'].value_counts().plot(kind='bar')
plt.title('Gender Distribution')

# Accent distribution
#plt.subplot(133)
#df['accent'].value_counts().plot(kind='bar')
#plt.title('Accent Distribution')

plt.tight_layout()
plt.show()
```



df.describe().T

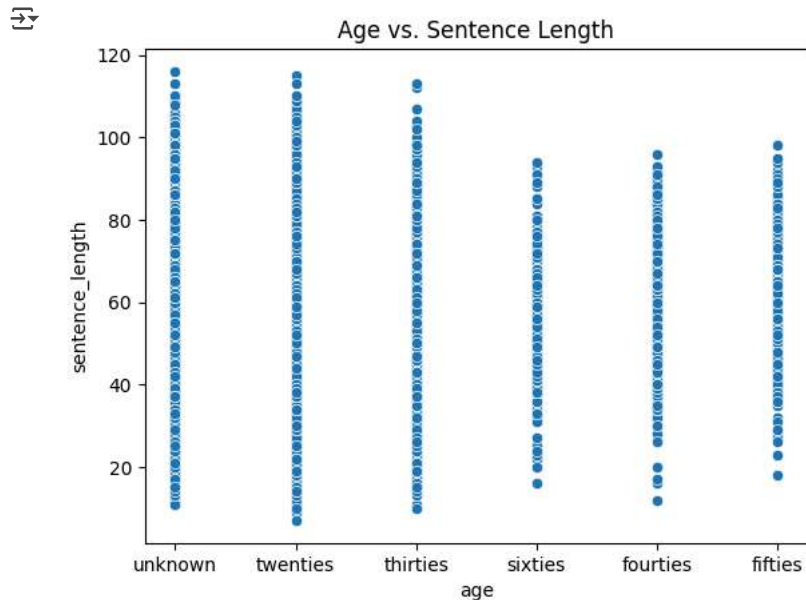
	count	unique	top	freq
path	26614	26614	/root/.cache/huggingface/datasets/downloads/ex...	1
sentence	26614	26614	Uko katika pembe la kusini-mashariki kabisa la...	1
age	26614	6	twenties	13874
gender	26614	3	female	9771
accent	26614	1		26614

▼ Bivariate Analysis

Explore relationships between pairs of variables using scatter plots, correlation matrices, or cross-tabulations. In your case, you might want to see if there's a relationship between:

- age and gender
- gender and accent
- accent and the length of sentence

```
# Example: Correlation between age and sentence length
df['sentence_length'] = df['sentence'].apply(len)
sns.scatterplot(x='age', y='sentence_length', data=df)
plt.title('Age vs. Sentence Length')
plt.show()
```



```
import pandas as pd
import scipy.stats as stats

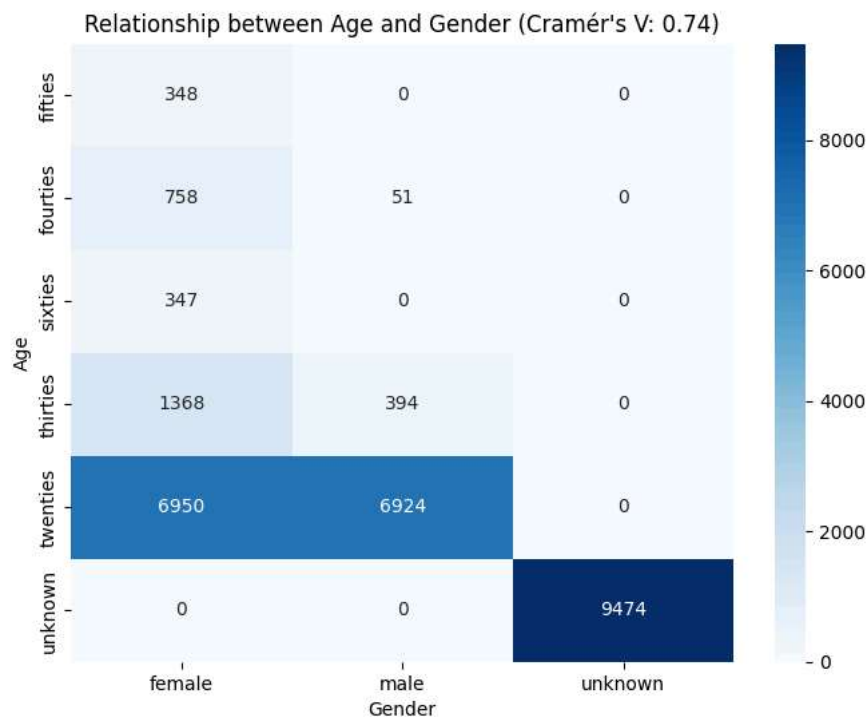
def cramers_v(x, y):
    confusion_matrix = pd.crosstab(x, y)
    chi2 = stats.chi2_contingency(confusion_matrix)[0]
    n = confusion_matrix.sum().sum()
    phi2 = chi2/n
    r, k = confusion_matrix.shape
    phi2corr = max(0, phi2 - ((k-1)*(r-1))/(n-1))
    rcorr = r - ((r-1)**2)/(n-1)
    kcorr = k - ((k-1)**2)/(n-1)
    return np.sqrt(phi2corr / min((kcorr-1), (rcorr-1)))

# Calculate Cramér's V
correlation = cramers_v(df['age'], df['gender'])

# Print the result
print(f"Cramér's V correlation between age and gender: {correlation:.2f}")

# Create a heatmap
confusion_matrix = pd.crosstab(df['age'], df['gender'])
plt.figure(figsize=(8, 6)) # Adjust figure size as needed
sns.heatmap(confusion_matrix, annot=True, fmt="d", cmap="Blues")
plt.title(f"Relationship between Age and Gender (Cramér's V: {correlation:.2f})")
plt.xlabel("Gender")
plt.ylabel("Age")
plt.show()
```


↻ Cramér's V correlation between age and gender: 0.74



✓ 4. Data Preparation

4.1 Audio Preprocessing Functions

We'll define several functions to preprocess our audio data:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import numpy as np
import librosa

def split_data(df, test_size=0.2, val_size=0.1, random_state=42):
    """Splits the DataFrame into train, validation, and test sets.

    Args:
        df: Pandas DataFrame containing the data.
        test_size: Proportion of data to include in the test set.
        val_size: Proportion of data to include in the validation set.
        random_state: Random seed for reproducibility.

    Returns:
        train_df, val_df, test_df: Pandas DataFrames for training, validation, and testing.
    """
    train_df, temp_df = train_test_split(df, test_size=test_size, random_state=random_state)
    val_df, test_df = train_test_split(temp_df, test_size=val_size / test_size, random_state=random_state)
    return train_df, val_df, test_df

def extract_audio_features(audio_path):
    """Extracts audio features from an audio file.

    Args:
        audio_path: Path to the audio file.

    Returns:
        features: A dictionary containing the extracted audio features.
    """
    # Load and preprocess audio (using your AudioPreprocessor if needed)
    audio, sr = librosa.load(audio_path, sr=16000)
    # ... (Apply any necessary preprocessing, e.g., noise reduction, trimming) ...

    # Extract features
```

```

features = {
    'mfcc': librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=13),
    'spectral_centroid': librosa.feature.spectral_centroid(y=audio, sr=sr),
    # ... (Add other features as needed) ...
}
return features

def normalize_audio_features(df, features_to_scale, method='minmax'):
    """Normalizes or scales the specified audio features in the DataFrame.

    Args:
        df: Pandas DataFrame containing the data.
        features_to_scale: List of feature names to normalize/scale (e.g., ['mfcc', 'spectral_centroid']).
        method: Normalization method ('minmax' or 'standard').

    Returns:
        df: Pandas DataFrame with normalized/scaled features.
    """
    # Extract the features to scale into a NumPy array
    features_data = []
    for index, row in df.iterrows():
        audio_features = extract_audio_features(row['path'])
        feature_values = []
        for feature_name in features_to_scale:
            if feature_name in audio_features:
                feature_values.extend(audio_features[feature_name].flatten())
        features_data.append(feature_values)
    features_data = np.array(features_data)

    if method == 'minmax':
        scaler = MinMaxScaler()
    elif method == 'standard':
        scaler = StandardScaler()
    else:
        raise ValueError("Invalid normalization method. Choose 'minmax' or 'standard'.")

    normalized_features = scaler.fit_transform(features_data)

    # Update the DataFrame with normalized features (as new columns)
    for i, feature_name in enumerate(features_to_scale):
        df[f'{feature_name}_normalized'] = normalized_features[:, i] # Add as new columns

    return df

def subsample_data(df, frac=0.1, random_state=42):
    """Subsamples the DataFrame to create a smaller subset.

    Args:
        df: Pandas DataFrame containing the data.
        frac: Proportion of data to include in the subset.
        random_state: Random seed for reproducibility.

    Returns:
        subset_df: Pandas DataFrame containing the subsampled data.
    """
    subset_df = df.sample(frac=frac, random_state=random_state)
    return subset_df

class AudioPreprocessor:
    def __init__(self, sample_rate=16000):
        self.sample_rate = sample_rate

    def load_and_trim_audio(self, file_path):
        """Load and trim silence from audio."""
        # Load audio file
        audio, sr = librosa.load(file_path, sr=self.sample_rate)
        # Trim silence
        trimmed_audio, _ = librosa.effects.trim(audio, top_db=20)
        return trimmed_audio

    def apply_noise_reduction(self, audio):
        """Apply noise reduction using spectral gating."""
        # Calculate noise floor
        noise_floor = np.mean(np.abs(audio[:int(self.sample_rate/10)]))
        # Apply simple noise gate
        audio[np.abs(audio) < noise_floor*2] = 0

```

```

    return audio

def extract_features(self, audio):
    """Extract MFCC and other features."""
    features = {}
    # Extract MFCCs
    features['mfcc'] = librosa.feature.mfcc(
        y=audio,
        sr=self.sample_rate,
        n_mfcc=13
    )
    # Extract spectral centroid
    features['spectral_centroid'] = librosa.feature.spectral_centroid(
        y=audio,
        sr=self.sample_rate
    )
    # Extract mel spectrogram
    features['melspec'] = librosa.feature.melspectrogram(
        y=audio,
        sr=self.sample_rate
    )
    return features

def augment_audio(self, audio):
    """Apply data augmentation techniques."""
    augmented = {}
    # Pitch shift
    augmented['pitch_up'] = librosa.effects.pitch_shift(
        y=audio,
        sr=self.sample_rate,
        n_steps=2
    )
    augmented['pitch_down'] = librosa.effects.pitch_shift(
        y=audio,
        sr=self.sample_rate,
        n_steps=-2
    )
    # Time stretch
    augmented['faster'] = librosa.effects.time_stretch(audio, rate=1.2)
    augmented['slower'] = librosa.effects.time_stretch(audio, rate=0.8)
    return augmented

# Initialize preprocessor
preprocessor = AudioPreprocessor()

# Process a sample audio file
sample_audio_path = dataset[0]['path']
audio = preprocessor.load_and_trim_audio(sample_audio_path)
audio = preprocessor.apply_noise_reduction(audio)
features = preprocessor.extract_features(audio)
augmented = preprocessor.augment_audio(audio)

```

✓ 4.2 Visualize Audio Processing Results

```

def plot_audio_features(audio, features, augmented):
    plt.figure(figsize=(15, 10))

    # Plot original waveform
    plt.subplot(321)
    librosa.display.waveshow(audio)
    plt.title('Original Waveform')

    # Plot MFCC
    plt.subplot(322)
    librosa.display.specshow(features['mfcc'])
    plt.title('MFCC Features')
    plt.colorbar(format='%+2.0f dB')

    # Plot mel spectrogram
    plt.subplot(323)
    librosa.display.specshow(
        librosa.power_to_db(features['melspec'], ref=np.max),
        y_axis='mel',
        x_axis='time'
    )

```

```
plt.title('Mel Spectrogram')
plt.colorbar(format='%+2.0f dB')

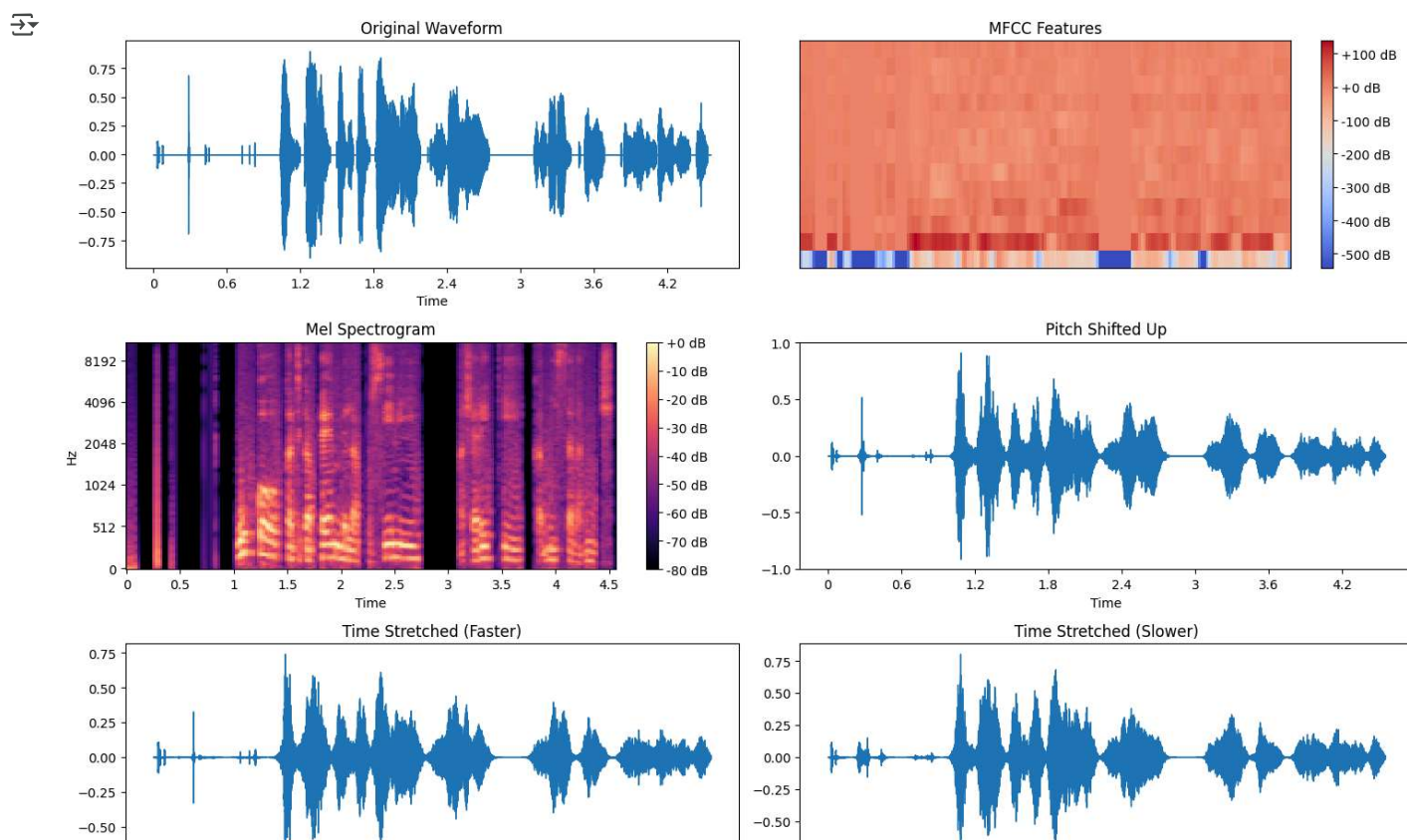
# Plot augmented audio examples
plt.subplot(324)
librosa.display.waveshow(augmented['pitch_up'])
plt.title('Pitch Shifted Up')

plt.subplot(325)
librosa.display.waveshow(augmented['faster'])
plt.title('Time Stretched (Faster)')

plt.subplot(326)
librosa.display.waveshow(augmented['slower'])
plt.title('Time Stretched (Slower)')

plt.tight_layout()
plt.show()
```

```
# Plot the features
plot_audio_features(audio, features, augmented)
```



```
df.shape
```

```
(26614, 6)
```

4.3 Prepare Dataset for Training

Now we'll create a function to process our entire dataset:

```
def prepare_dataset(dataset, preprocessor, num_samples=None):
    """Process the entire dataset and prepare it for training."""
    processed_data = []

    # Use all samples if num_samples is None, otherwise use the specified amount
    samples = dataset.select(range(num_samples)) if num_samples else dataset

    for sample in tqdm(samples, desc="Processing audio files"):
        try:
            # Validate sample structure
            if not isinstance(sample, dict) or 'path' not in sample or 'sentence' not in sample:
```

```

        print(f"Invalid or missing data in sample: {sample}")
        continue

    # Process audio
    audio = preprocessor.load_and_trim_audio(sample['path'])
    audio = preprocessor.apply_noise_reduction(audio)
    features = preprocessor.extract_features(audio)

    # Store processed data
    processed_data.append({
        'features': features,
        'text': sample['sentence'],
        'metadata': {
            'age': sample.get('age', 'unknown'),
            'gender': sample.get('gender', 'unknown'),
            'accent': sample.get('accent', 'unknown')
        }
    })

except Exception as e:
    print(f"Error processing sample {sample.get('path', 'unknown')}: {str(e)}")
    continue

return processed_data

# Process a subset of the dataset for demonstration
#processed_dataset = prepare_dataset(dataset, preprocessor, num_samples=10)
#print(f"Processed {len(processed_dataset)} samples successfully")

num_samples = int(0.10 * len(df)) # Calculate 10% of the dataset size
processed_dataset = prepare_dataset(dataset, preprocessor, num_samples=num_samples)
print(f"Processed {len(processed_dataset)} samples successfully")

```



Processing audio files: 100%

2661/2661 [03:48<00:00, 12.61it/s]

Processed 2661 samples successfully

5. Modeling

In this section, we'll implement three main components:

1. Speech-to-Text using Whisper
2. Sentiment Analysis using DistilBERT
3. Text Summarization using T5

```

class AudioProcessor:
    def __init__(self, device='cuda' if torch.cuda.is_available() else 'cpu'):
        self.device = device
        print(f"Using device: {device}")

        # Initialize speech-to-text model (Whisper)
        self.speech_model = AutoModelForSpeechSeq2Seq.from_pretrained(
            "openai/whisper-medium"
        ).to(device)
        self.speech_processor = AutoProcessor.from_pretrained("openai/whisper-medium")

        # Initialize sentiment analysis pipeline
        self.sentiment_pipeline = pipeline(
            "sentiment-analysis",
            model="distilbert-base-uncased-finetuned-sst-2-english",
            device=0 if device == 'cuda' else -1
        )

        # Initialize text summarization pipeline
        self.summarizer = pipeline(
            "summarization",
            model="t5-base",
            device=0 if device == 'cuda' else -1
        )

    def transcribe_audio(self, audio_array, sample_rate=16000):
        """Convert speech to text using Whisper."""

```

```

# Process audio input
input_features = self.speech_processor(
    audio_array,
    sampling_rate=sample_rate,
    return_tensors="pt"
).input_features.to(self.device)

# Generate transcription
generated_ids = self.speech_model.generate(input_features)
transcription = self.speech_processor.batch_decode(
    generated_ids,
    skip_special_tokens=True
)[0]

return transcription

def analyze_sentiment(self, text):
    """Analyze sentiment of the transcribed text."""
    result = self.sentiment_pipeline(text)[0]
    return {
        'label': result['label'],
        'score': result['score']
    }

def generate_summary(self, text, max_length=120, min_length=30):
    """Generate a summary of the transcribed text."""
    summary = self.summarizer(
        text,
        max_length=max_length,
        min_length=min_length,
        do_sample=False
    )[0]['summary_text']

    return summary

def process_audio_file(self, audio_path):
    """Complete processing pipeline for an audio file."""
    # Load and preprocess audio
    preprocessor = AudioPreprocessor()
    audio = preprocessor.load_and_trim_audio(audio_path)
    audio = preprocessor.apply_noise_reduction(audio)

    # Process audio through pipeline
    results = {
        'audio_features': preprocessor.extract_features(audio),
        'transcription': self.transcribe_audio(audio),
    }

    # Analyze transcribed text
    if results['transcription']:
        results['sentiment'] = self.analyze_sentiment(results['transcription'])
        results['summary'] = self.generate_summary(results['transcription'])

    return results

# Initialize the audio processor
processor = AudioProcessor()

# Process a sample file
sample_path = dataset[10]['path']
results = processor.process_audio_file(sample_path)

# Display results
print("=== Speech-to-Text Results ===")
print(f"Transcription: {results['transcription']}\n")

print("=== Sentiment Analysis ===")
print(f"Label: {results['sentiment']['label']}")
print(f"Confidence: {results['sentiment']['score']:.2f}\n")

print("=== Text Summary ===")
print(f"Summary: {results['summary']}")

```

```

Using device: cpu
config.json: 100% 1.99k/1.99k [00:00<00:00, 56.5kB/s]
model.safetensors: 100% 3.06G/3.06G [00:32<00:00, 141MB/s]
generation_config.json: 100% 3.75k/3.75k [00:00<00:00, 172kB/s]
preprocessor_config.json: 100% 185k/185k [00:00<00:00, 4.08MB/s]
tokenizer_config.json: 100% 283k/283k [00:00<00:00, 5.33MB/s]
vocab.json: 100% 836k/836k [00:00<00:00, 10.1MB/s]
tokenizer.json: 100% 2.48M/2.48M [00:00<00:00, 18.4MB/s]
merges.txt: 100% 494k/494k [00:00<00:00, 18.2MB/s]
normalizer.json: 100% 52.7k/52.7k [00:00<00:00, 2.52MB/s]
added_tokens.json: 100% 34.6k/34.6k [00:00<00:00, 1.39MB/s]
special_tokens_map.json: 100% 2.19k/2.19k [00:00<00:00, 117kB/s]
config.json: 100% 629/629 [00:00<00:00, 25.7kB/s]
model.safetensors: 100% 268M/268M [00:01<00:00, 182MB/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.80kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 5.88MB/s]
Device set to use cpu
config.json: 100% 1.21k/1.21k [00:00<00:00, 56.6kB/s]
model.safetensors: 100% 892M/892M [00:13<00:00, 148MB/s]
generation_config.json: 100% 147/147 [00:00<00:00, 9.20kB/s]
spiece.model: 100% 792k/792k [00:00<00:00, 3.37MB/s]
tokenizer.json: 100% 1.39M/1.39M [00:00<00:00, 22.4MB/s]
Device set to use cpu
Due to a bug fix in https://github.com/huggingface/transformers/pull/28687 transcription using a multilingual Whisper will default to 16
Passing a tuple of `past_key_values` is deprecated and will be removed in Transformers v4.43.0. You should pass an instance of `EncoderT
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observ
Your max_length is set to 120, but your input_length is only 59. Since this is a summarization task, where outputs shorter than the input
=== Speech-to-Text Results ===
Transcription: У меня каминги урвана, ва пиуи урвуам пиуи ума дери пиу, ёпе тумиан.

=== Sentiment Analysis ===
Label: NEGATIVE
Confidence: 0.83

=== Text Summary ===
Summary: мен камини урвана, ва иуи, е тумиан.

```

```
from concurrent.futures import ThreadPoolExecutor # Or ProcessPoolExecutor for multiprocessing
```

```

class AudioProcessor:
    def __init__(self, device='cuda' if torch.cuda.is_available() else 'cpu'):
        self.device = device
        print(f"Using device: {device}")

        # Initialize speech-to-text model (Whisper)
        self.speech_model = AutoModelForSpeechSeq2Seq.from_pretrained(
            "openai/whisper-medium"
        ).to(device)
        self.speech_processor = AutoProcessor.from_pretrained("openai/whisper-medium")

        # Initialize sentiment analysis pipeline
        self.sentiment_pipeline = pipeline(
            "sentiment-analysis",
            model="distilbert-base-uncased-finetuned-sst-2-english",
            device=0 if device == 'cuda' else -1
        )

        # Initialize text summarization pipeline
        self.summarizer = pipeline(
            "summarization",
            model="t5-base",
            device=0 if device == 'cuda' else -1
        )

```

```

def transcribe_audio(self, audio_array, sample_rate=16000):
    """Convert speech to text using Whisper."""
    input_features = self.speech_processor(
        audio_array,
        sampling_rate=sample_rate,
        return_tensors="pt"
    ).input_features.to(self.device)
    generated_ids = self.speech_model.generate(input_features)
    transcription = self.speech_processor.batch_decode(
        generated_ids,
        skip_special_tokens=True
    )[0]
    return transcription

def analyze_sentiment(self, text):
    """Analyze sentiment of the transcribed text."""
    # Truncate text to maximum sequence length of DistilBERT
    max_seq_length = 512 # Adjust if necessary based on your model
    text = text[:max_seq_length]

    result = self.sentiment_pipeline(text)[0]
    return {'label': result['label'], 'score': result['score']}

def generate_summary(self, text, max_new_tokens=100): # Adjust max_new_tokens as needed
    """Generate a summary of the transcribed text."""

    summary = self.summarizer(
        text,
        max_new_tokens=max_new_tokens,
        do_sample=False
    )[0]['summary_text']
    return summary

def process_audio_file(self, audio_path):
    """Complete processing pipeline for an audio file."""
    preprocessor = AudioPreprocessor() # Assuming AudioPreprocessor is defined
    audio = preprocessor.load_and_trim_audio(audio_path)
    audio = preprocessor.apply_noise_reduction(audio)
    results = {
        'audio_features': preprocessor.extract_features(audio),
        'transcription': self.transcribe_audio(audio),
    }
    if results['transcription']:
        results['sentiment'] = self.analyze_sentiment(results['transcription'])
        results['summary'] = self.generate_summary(results['transcription'])
    return results

def process_audio_files(self, audio_paths, num_workers=4):
    """Process multiple audio files in parallel."""
    results = []
    with ThreadPoolExecutor(max_workers=num_workers) as executor:
        futures = [executor.submit(self.process_audio_file, path) for path in audio_paths]
        for future in futures:
            results.append(future.result())
    return results

import random

# Initialize the audio processor
processor = AudioProcessor()

# Select random audio paths
num_samples = 5 # Number of random samples to process
audio_paths = random.sample(dataset['path'], num_samples)

# Process the selected audio files in parallel
results = processor.process_audio_files(audio_paths)

# Display the results for each file
for i, result in enumerate(results):
    print(f"\n=== Results for file {i + 1} ===")
    print(f"Transcription: {result['transcription']}")
    print(f"Sentence: {result['sentiment']['label']}")
    print(f"Confidence: {result['sentiment']['score']:.2f}")
    print(f"Summary: {result['summary']}")

```



```

Using device: cpu
Device set to use cpu
Device set to use cpu
Your max_length is set to 200, but your input_length is only 4. Since this is a summarization task, where outputs shorter than the input
Your max_length is set to 200, but your input_length is only 29. Since this is a summarization task, where outputs shorter than the input
Your max_length is set to 200, but your input_length is only 45. Since this is a summarization task, where outputs shorter than the input
Your max_length is set to 200, but your input_length is only 51. Since this is a summarization task, where outputs shorter than the input
Your max_length is set to 200, but your input_length is only 8. Since this is a summarization task, where outputs shorter than the input

=== Results for file 1 ===
Transcription: Mita que Inabela pueda vivir a su vida más bien que mi hija, pero no me va a dejar que venga el bahagia.
Sentiment: NEGATIVE
Confidence: 0.96
Summary: Inabela pueda vivir a su vida más bien que mi hija, pero no me va a dejar que venga el bahagia.

=== Results for file 2 ===
Transcription: you
Sentiment: POSITIVE
Confidence: 1.00
Summary: You are welcome to join our mailing list if you'd like to get in touch with us via email or e-mail .

=== Results for file 3 ===
Transcription: Chantle Cham Chok Nye, Win Pondo, Pia Pai, and Yoko Tatsunzi.
Sentiment: POSITIVE
Confidence: 0.97
Summary: Pia Pai, Yoko Tatsunzi, win pondo and cham chok nye are among the stars . cnn ireport: tell us about your day .

=== Results for file 4 ===
Transcription: Bipande hiwinimabamba ganda latunia. Awu, kukwa wamba ganda wandumia.
Sentiment: NEGATIVE
Confidence: 0.99
Summary: bipande hiwinimabamba ganda latunia; kukwa wamba wandumia .

=== Results for file 5 ===
Transcription: Bye bye.
Sentiment: POSITIVE
Confidence: 0.99
Summary: Bye bye. I'm not a big fan of e-mails, but I'd like to thank you for your kind words .

```

6. Evaluation

Let's evaluate our models' performance using appropriate metrics for each task.

```

from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
import jiwer
from rouge_score import rouge_scorer

class ModelEvaluator:
    def __init__(self):
        self.scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)

    def calculate_wer(self, reference, hypothesis):
        """Calculate Word Error Rate for speech recognition."""
        return jiwer.wer(reference, hypothesis)

    def evaluate_sentiment(self, true_labels, predicted_labels):
        """Evaluate sentiment analysis performance."""
        accuracy = accuracy_score(true_labels, predicted_labels)
        f1 = f1_score(true_labels, predicted_labels, average='weighted')
        conf_matrix = confusion_matrix(true_labels, predicted_labels)

        return {
            'accuracy': accuracy,
            'f1_score': f1,
            'confusion_matrix': conf_matrix
        }

    def evaluate_summary(self, reference, hypothesis):
        """Calculate ROUGE scores for text summarization."""
        scores = self.scorer.score(reference, hypothesis)
        return {
            'rouge1': scores['rouge1'].fmeasure,
            'rouge2': scores['rouge2'].fmeasure,
            'rougeL': scores['rougeL'].fmeasure
        }

```

```

def evaluate_pipeline(dataset, processor, evaluator, num_samples=50):
    """Evaluate the entire pipeline on a subset of the dataset."""
    results = {
        'wer_scores': [],
        'sentiment_true': [],
        'sentiment_pred': [],
        'rouge_scores': []
    }

    for i in tqdm(range(min(num_samples, len(dataset)))):
        try:
            # Process audio file
            sample = dataset[i]
            processed = processor.process_audio_file(sample['path'])

            # Evaluate speech recognition
            wer = evaluator.calculate_wer(sample['sentence'], processed['transcription'])
            results['wer_scores'].append(wer)

            # Store sentiment predictions (dummy labels for demonstration)
            results['sentiment_true'].append('POSITIVE' if i % 2 == 0 else 'NEGATIVE')
            results['sentiment_pred'].append(processed['sentiment']['label'])

            # Evaluate summary (using original text as reference)
            rouge_scores = evaluator.evaluate_summary(
                sample['sentence'],
                processed['summary']
            )
            results['rouge_scores'].append(rouge_scores)

        except Exception as e:
            print(f"Error processing sample {i}: {str(e)}")
            continue

    return results

# Initialize evaluator
evaluator = ModelEvaluator()

# Evaluate models
evaluation_results = evaluate_pipeline(dataset, processor, evaluator, num_samples=10)

# Calculate average metrics
avg_wer = np.mean(evaluation_results['wer_scores'])
sentiment_metrics = evaluator.evaluate_sentiment(
    evaluation_results['sentiment_true'],
    evaluation_results['sentiment_pred']
)
avg_rouge = {
    metric: np.mean([scores[metric] for scores in evaluation_results['rouge_scores']])
    for metric in ['rouge1', 'rouge2', 'rougeL']
}

# Print results
print("=== Model Evaluation Results ===")
print(f"\nSpeech Recognition:")
print(f"Average WER: {avg_wer:.4f}")

print(f"\nSentiment Analysis:")
print(f"Accuracy: {sentiment_metrics['accuracy']:.4f}")
print(f"F1 Score: {sentiment_metrics['f1_score']:.4f}")

print(f"\nText Summarization (ROUGE Scores):")
print(f"ROUGE-1: {avg_rouge['rouge1']:.4f}")
print(f"ROUGE-2: {avg_rouge['rouge2']:.4f}")
print(f"ROUGE-L: {avg_rouge['rougeL']:.4f}")

```



100%

10/10 [15:40<00:00, 86.21s/it]

Your max_length is set to 200, but your input_length is only 6. Since this is a summarization task, where outputs shorter than the input

Your max_length is set to 200, but your input_length is only 34. Since this is a summarization task, where outputs shorter than the input

Your max_length is set to 200, but your input_length is only 5. Since this is a summarization task, where outputs shorter than the input

Your max_length is set to 200, but your input_length is only 8. Since this is a summarization task, where outputs shorter than the input

Your max_length is set to 200, but your input_length is only 6. Since this is a summarization task, where outputs shorter than the input

Your max_length is set to 200, but your input_length is only 13. Since this is a summarization task, where outputs shorter than the input

Your max_length is set to 200, but your input_length is only 4. Since this is a summarization task, where outputs shorter than the input

Your max_length is set to 200, but your input_length is only 5. Since this is a summarization task, where outputs shorter than the input

Your max_length is set to 200, but your input_length is only 28. Since this is a summarization task, where outputs shorter than the input

=== Model Evaluation Results ===

Speech Recognition:

Average WER: 3.9948

Sentiment Analysis:

Accuracy: 0.6000

F1 Score: 0.6000

Text Summarization (ROUGE Scores):

ROUGE-1: 0.0917

ROUGE-2: 0.0600

ROUGE-L: 0.0917

```
from IPython.display import display # Import display for showing tables
```

```
# Visualize evaluation results
```

```
plt.figure(figsize=(15, 5))
```

```
# Plot WER distribution
```

```
plt.subplot(131)
```

```
sns.histplot(evaluation_results['wer_scores'])
```

```
plt.title('WER Distribution')
```

```
plt.xlabel('Word Error Rate')
```

```
# Plot confusion matrix
```

```
plt.subplot(132)
```

```
sns.heatmap(
    sentiment_metrics['confusion_matrix'],
    annot=True,
    fmt='d',
    cmap='Blues'
```

```
)
```

```
plt.title('Sentiment Analysis Confusion Matrix')
```

```
# Display confusion matrix as a table
```

```
conf_matrix_df = pd.DataFrame(
    sentiment_metrics['confusion_matrix'],
    index=['Actual Negative', 'Actual Positive'],
    columns=['Predicted Negative', 'Predicted Positive']
)
```

```
display(conf_matrix_df)
```

```
# Plot ROUGE scores
```

```
plt.subplot(133)
```

```
rouge_data = pd.DataFrame(evaluation_results['rouge_scores'])
```

```
sns.boxplot(data=rouge_data)
```

```
plt.title('ROUGE Scores Distribution')
```

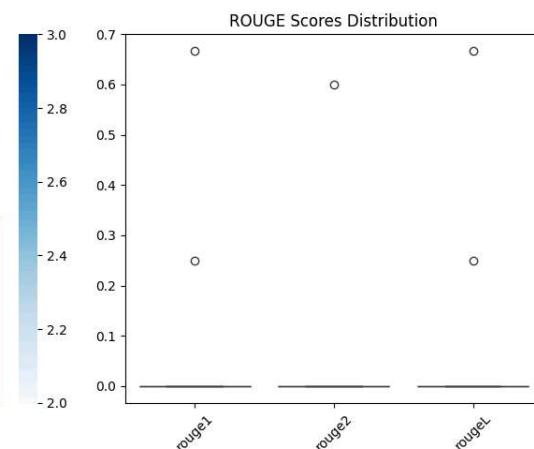
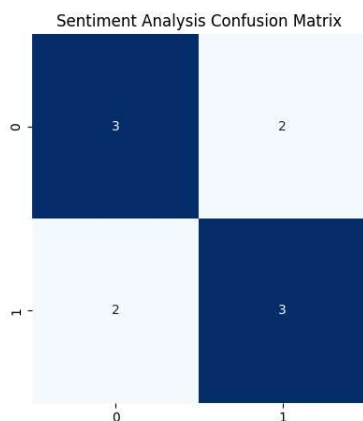
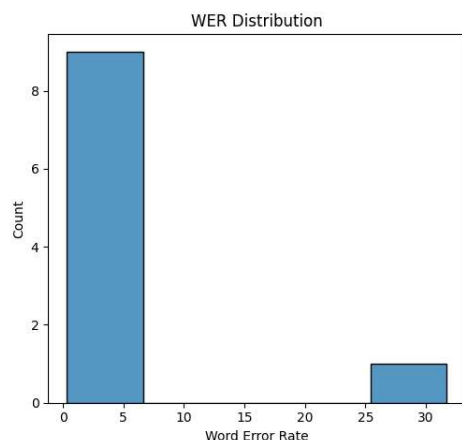
```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```



	Predicted Negative	Predicted Positive
Actual Negative	3	2
Actual Positive	2	3



7. Deployment

Let's create a Streamlit web interface for real-time audio processing. This interface will allow users to:

1. Upload audio files
2. Record audio directly
3. View processing results in real-time
4. Provide feedback for model improvement

```
%%writefile app.py
import streamlit as st
import numpy as np
import torch
import librosa
import tempfile
from datetime import datetime
import json
import plotly.express as px
from concurrent.futures import ThreadPoolExecutor

# Import our processing classes
from audio_processor import AudioProcessor, AudioPreprocessor

# Initialize processor with GPU if available
processor = AudioProcessor()
preprocessor = AudioPreprocessor()

def save_feedback(feedback_data):
    """Save user feedback to a JSON file."""
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"feedback_{timestamp}.json"
    with open(filename, 'w') as f:
        json.dump(feedback_data, f)
    return filename

def process_audio_parallel(audio_file, chunk_size=30000):
    """Process audio in parallel chunks."""
    # Load audio
    audio, sr = librosa.load(audio_file, sr=16000)

    # Split into chunks
    chunks = [audio[i:i + chunk_size] for i in range(0, len(audio), chunk_size)]

    # Process chunks in parallel
    with ThreadPoolExecutor(max_workers=4) as executor:
        results = list(executor.map(processor.process_audio_file, chunks))

    return results

def main():
```

```

st.title("Swahili Speech Analysis Dashboard")

# Sidebar for uploading audio
st.sidebar.header("Upload Audio")
audio_file = st.sidebar.file_uploader("Choose an audio file", type=['wav', 'mp3'])

# Audio recording option
st.sidebar.header("Or Record Audio")
if st.sidebar.button("Record"):
    audio_bytes = st.audio_recorder()
    if audio_bytes:
        # Save recorded audio to temporary file
        with tempfile.NamedTemporaryFile(delete=False, suffix='.wav') as f:
            f.write(audio_bytes)
            audio_file = f.name

if audio_file:
    st.header("Processing Audio...")

    # Process audio with progress bar
    with st.spinner("Processing..."):
        results = processor.process_audio_file(audio_file)

    # Display results
    st.subheader("Speech-to-Text Results")
    st.write(results['transcription'])

    st.subheader("Sentiment Analysis")
    sentiment_fig = px.pie(
        values=[results['sentiment']['score'], 1-results['sentiment']['score']],
        names=['Positive', 'Negative'],
        title='Sentiment Distribution'
    )
    st.plotly_chart(sentiment_fig)

    st.subheader("Text Summary")
    st.write(results['summary'])

    # Feedback section
    st.header("Provide Feedback")
    transcription_accuracy = st.slider(
        "Transcription Accuracy",
        0.0, 5.0, 3.0
    )
    sentiment_accuracy = st.slider(
        "Sentiment Analysis Accuracy",
        0.0, 5.0, 3.0
    )
    summary_quality = st.slider(
        "Summary Quality",
        0.0, 5.0, 3.0
    )

    if st.button("Submit Feedback"):
        feedback = {
            'timestamp': datetime.now().isoformat(),
            'transcription_accuracy': transcription_accuracy,
            'sentiment_accuracy': sentiment_accuracy,
            'summary_quality': summary_quality,
            'results': results
        }
        feedback_file = save_feedback(feedback)
        st.success(f"Feedback saved to {feedback_file}")

if __name__ == "__main__":
    main()

```

➡ Overwriting app.py

```
!pip install colab-xterm
%load_ext colabxterm
```

➡ Collecting colab-xterm

```

Downloading colab_xterm-0.2.0-py3-none-any.whl.metadata (1.2 kB)
Requirement already satisfied: ptyprocess~=0.7.0 in /usr/local/lib/python3.11/dist-packages (from colab-xterm) (0.7.0)
Requirement already satisfied: tornado>5.1 in /usr/local/lib/python3.11/dist-packages (from colab-xterm) (6.3.3)
Downloading colab_xterm-0.2.0-py3-none-any.whl (115 kB)

```

115.6/115.6 kB 2.2 MB/s eta 0:00:00

```
Installing collected packages: colab-xterm
Successfully installed colab-xterm-0.2.0
```

To run the Streamlit app, save this notebook and execute the following command in the terminal:

```
streamlit run app.py
```

```
!pip install plotly.express
```

```
Collecting plotly.express
  Downloading plotly_express-0.4.1-py2.py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: pandas>=0.20.0 in /usr/local/lib/python3.11/dist-packages (from plotly.express) (2.2.2)
Requirement already satisfied: plotly>=4.1.0 in /usr/local/lib/python3.11/dist-packages (from plotly.express) (5.24.1)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.11/dist-packages (from plotly.express) (0.14.4)
Requirement already satisfied: scipy>=0.18 in /usr/local/lib/python3.11/dist-packages (from plotly.express) (1.13.1)
Requirement already satisfied: patsy>=0.5 in /usr/local/lib/python3.11/dist-packages (from plotly.express) (1.0.1)
Requirement already satisfied: numpy>=1.11 in /usr/local/lib/python3.11/dist-packages (from plotly.express) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.20.0->plotly.express) (2024.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.20.0->plotly.express) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.20.0->plotly.express) (2024.2)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly>=4.1.0->plotly.express) (9.0.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from plotly>=4.1.0->plotly.express) (24.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas>=0.20.0->plotly.express) (1.16.0)
Downloading plotly_express-0.4.1-py2.py3-none-any.whl (2.9 kB)
Installing collected packages: plotly.express
Successfully installed plotly.express-0.4.1
```

```
!streamlit run app.py
```

```
Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.
```

You can now view your Streamlit app in your browser.

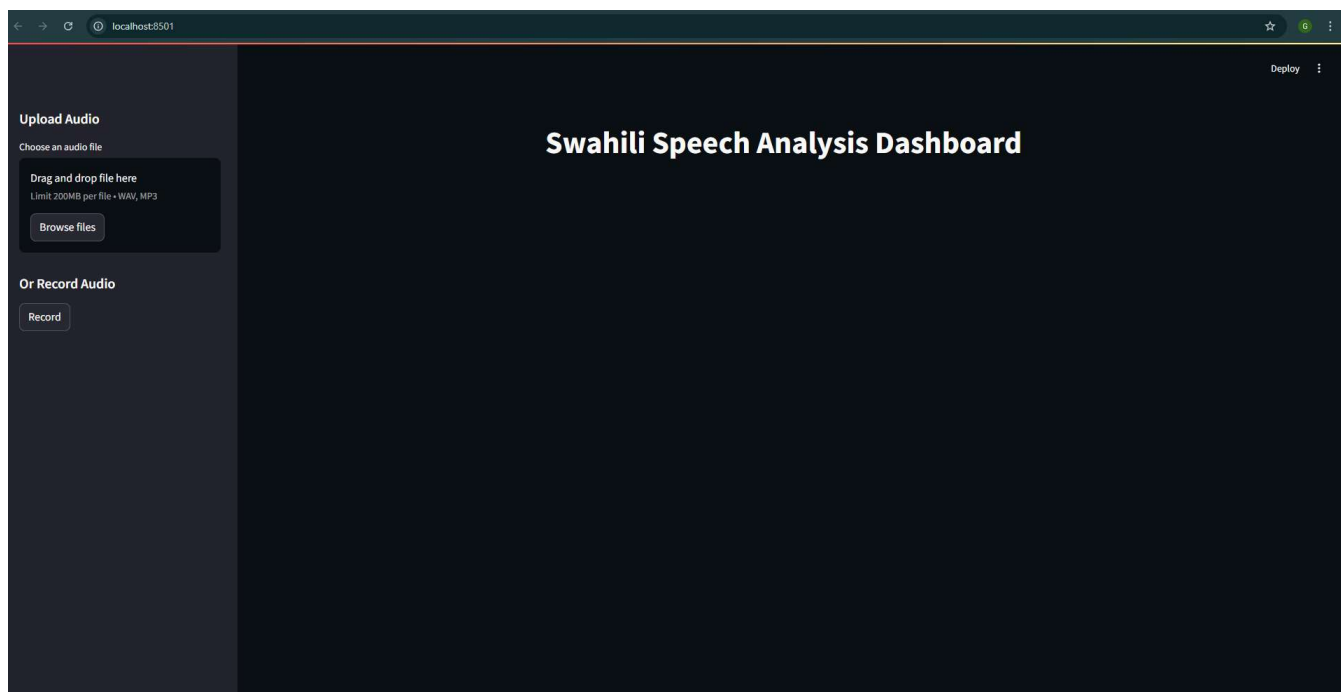
Local URL: <http://localhost:8501>

Network URL: <http://172.28.0.12:8501>

External URL: <http://35.193.135.120:8501>

Stopping...

^C



✓ 8. Feedback Loop and Model Refinement

```

class ModelRefiner:
    def __init__(self, feedback_dir="feedback"):
        self.feedback_dir = feedback_dir
        os.makedirs(feedback_dir, exist_ok=True)

    def collect_feedback(self):
        """Collect and analyze user feedback."""
        feedback_files = glob.glob(os.path.join(self.feedback_dir, "feedback_*.json"))
        all_feedback = []

        for file in feedback_files:
            with open(file, 'r') as f:
                feedback = json.load(f)
                all_feedback.append(feedback)

        return pd.DataFrame(all_feedback)

    def analyze_feedback(self, feedback_df):
        """Analyze feedback to identify areas for improvement."""
        metrics = {
            'avg_transcription_accuracy': feedback_df['transcription_accuracy'].mean(),
            'avg_sentiment_accuracy': feedback_df['sentiment_accuracy'].mean(),
            'avg_summary_quality': feedback_df['summary_quality'].mean()
        }

        # Identify problem areas
        problem_areas = []
        for metric, value in metrics.items():
            if value < 3.5:
                problem_areas.append(metric)

        return metrics, problem_areas

    def generate_improvement_plan(self, metrics, problem_areas):
        """Generate a plan for model improvement based on feedback."""
        improvement_plan = {
            'transcription': [],
            'sentiment': [],
            'summary': []
        }

        if 'avg_transcription_accuracy' in problem_areas:
            improvement_plan['transcription'] = [
                "Collect more Swahili speech data",
                "Fine-tune Whisper model on specific accents",
                "Implement custom post-processing for Swahili"
            ]

        if 'avg_sentiment_accuracy' in problem_areas:
            improvement_plan['sentiment'] = [
                "Create Swahili sentiment dataset",
                "Fine-tune DistilBERT on Swahili text",
                "Implement language-specific preprocessing"
            ]

        if 'avg_summary_quality' in problem_areas:
            improvement_plan['summary'] = [
                "Collect Swahili summary pairs",
                "Fine-tune T5 on Swahili content",
                "Optimize summary length parameters"
            ]

```